MAI 5301 - Week 13 Presentation

Daryl Nelson
USI: 1021215

WEB ARENA: A REALISTIC WEB ENVIRONMENT FOR BUILDING AUTONOMOUS AGENTS

Shuyan Zhou∗ Frank F. Xu∗ Hao Zhu † Xuhui Zhou† Robert Lo† Abishek Sridhar† Xianyi Cheng Tianyue Ou Yonatan Bisk Daniel Fried Uri Alon Graham Neubig

Autonomous Agents and Natural Language

Autonomous agents can potentially perform everyday web tasks using natural language instructions.

Existing testing environments are often simplified, static, or unrealistic.

This creates a gap between AI capabilities in research and real-world applications.

The paper introduces WebArena, a realistic environment to evaluate and develop web-based AI agents.

Importance of Realistic Evaluation Environments

Problems with previous environments:

- Many rely on static or simplified environments.
- Some use live websites, which change constantly and reduce reproducibility.
- Existing benchmarks often limit task complexity and exploration.

Goal of WebArena:

- Provide a realistic, controlled, and reproducible environment for evaluating autonomous agents.

Importance of Realistic Evaluation Environments

To properly evaluate AI agents, environments must be:

- Authentic (Realistic)
  - Similar to real-world systems and websites.

- Reproducible
  - Experiments can be repeated consistently by different researchers.

Why this matters:

- Allows fair comparison of AI systems.

- Helps measure progress on tasks humans actually perform online.

Problems with Existing Agent Environments

Many current evaluation environments have major limitations:

1. Over-simplified systems
    - Missing many features of real-world platforms.

2. Limited task diversity
    - Few types of tasks available.

3. Reduced task complexity
    - Real-world problems are much more complex.

Result:

- AI agents appear capable in experiments but struggle in real-world situations.

Static and Restricted Environments

Some environments are static.

Meaning:

- Agents interact only with pre-recorded states or pages.

Limitations:

- No dynamic exploration.
- No real interaction with systems.

Example analogy:

- Learning to browse a website using screenshots instead of the real website.

Issues with Current Evaluation Methods

Many evaluation methods compare:

- The text description of actions
- With a predefined reference action sequence

Problem:

- Real tasks often have multiple valid solutions.

Example:

- Two users may reach the same webpage using different navigation paths.

Better approach:

- Evaluate whether the task goal was actually completed

Introducing WebArena

Researchers propose WebArena.

Purpose:

- Provide a realistic and reproducible web environment for training and evaluating autonomous agents.

Key idea:

- Simulate real internet tasks in a controlled research environment.

WebArena Website Domains

WebArena includes four functional web platforms:

1. Online shopping websites
2. Discussion forums
3. Collaborative software development platforms
4. Business content management systems

These domains represent common activities people perform on the internet.

Tools and Knowledge Resources

WebArena includes additional tools to support problem solving:

- Map
- Calculator
- Scratchpad for notes

Knowledge sources include:

- General resources such as Wikipedia
- Domain-specific documentation and manuals.

Purpose:

- Encourage human-like reasoning and decision making.

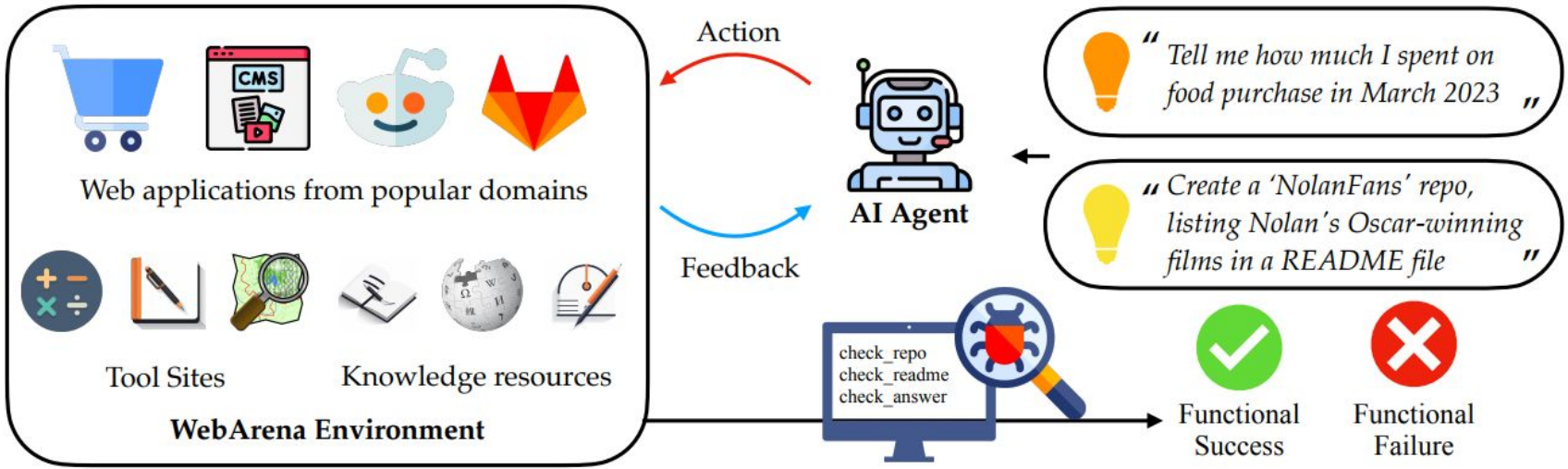


Figure 1: WebArena is a standalone, self-hostable web environment for building autonomous agents. WebArena creates websites from four popular categories with functionality and data mimicking their real-world equivalents. To emulate human problem-solving, WebArena also embeds tools and knowledge resources as independent websites. WebArena introduces a benchmark on interpreting *high-level realistic* natural language command to concrete web-based interactions. We provide validators to programmatically validate the functional correctness of each task.

Benchmark Task Dataset

WebArena includes 812 web-based tasks.

Characteristics:

- Long-horizon tasks (multiple steps required)
- Written as natural language instructions
- Simulates how humans give commands.

Example task:

- “Update documentation in a repository with installation instructions.”

Evaluation Method

The benchmark evaluates functional correctness.

Meaning:

- Check whether the task goal was achieved.

Example:

- If a repository file must be updated,
- The system verifies the actual content change, not just the actions performed.

Advantage:

- Supports multiple valid solutions.

Testing AI Agents

Researchers evaluated agents using powerful language models:

- GPT - 4
- PaLM 2

Agents were implemented using:

- Few-shot in-context learning
- Small examples to guide behavior.

WebArena Environment

Goal of WebArena:

- Create a realistic and reproducible web environment for testing AI agents.

Challenges with real websites:

- Websites constantly change.
- Bots may be blocked by CAPTCHAs.
- Content updates frequently.

Solution:

- Build a standalone simulated web environment that behaves like real websites.

Ensuring Reproducibility

WebArena avoids using live websites.

Benefits:

- Same environment for every experiment.
- Fair comparison between AI systems.
- No interference from:
  - CAPTCHAs
  - website updates
  - configuration changes

Result:

- Researchers can repeat experiments consistently.

Maintaining Realism

To keep the environment realistic:

- Use open-source libraries used by real websites.
- Import actual data from real-world platforms.
- Replicate common website features and structures.

Outcome:

- Environment behaves similarly to real internet platforms.

Agent Interaction Model

WebArena is represented as:

$E = \langle S, A, O, T \rangle$

Where:

- S (State Space) – current condition of the environment
- A (Action Space) – actions the agent can perform
- O (Observation Space) – information the agent sees
- T (Transition Function) – how actions change the state

Example:

- Click "Add to Cart"
- State changes → item appears in cart.

Natural Language Task Instructions

Tasks are given as high-level natural language instructions.

Example intent:

- “Purchase a laptop under $1000.”

Agent uses:

- Task instruction
- Current webpage observation
- Previous actions and observations

To decide the next action.

Reward Function

WebArena measures success using a reward function.

It checks:

- Whether the task goal was achieved
- Whether the actions were correct

Example:

Task: Place an order → Evaluation: Check if an actual order was placed.

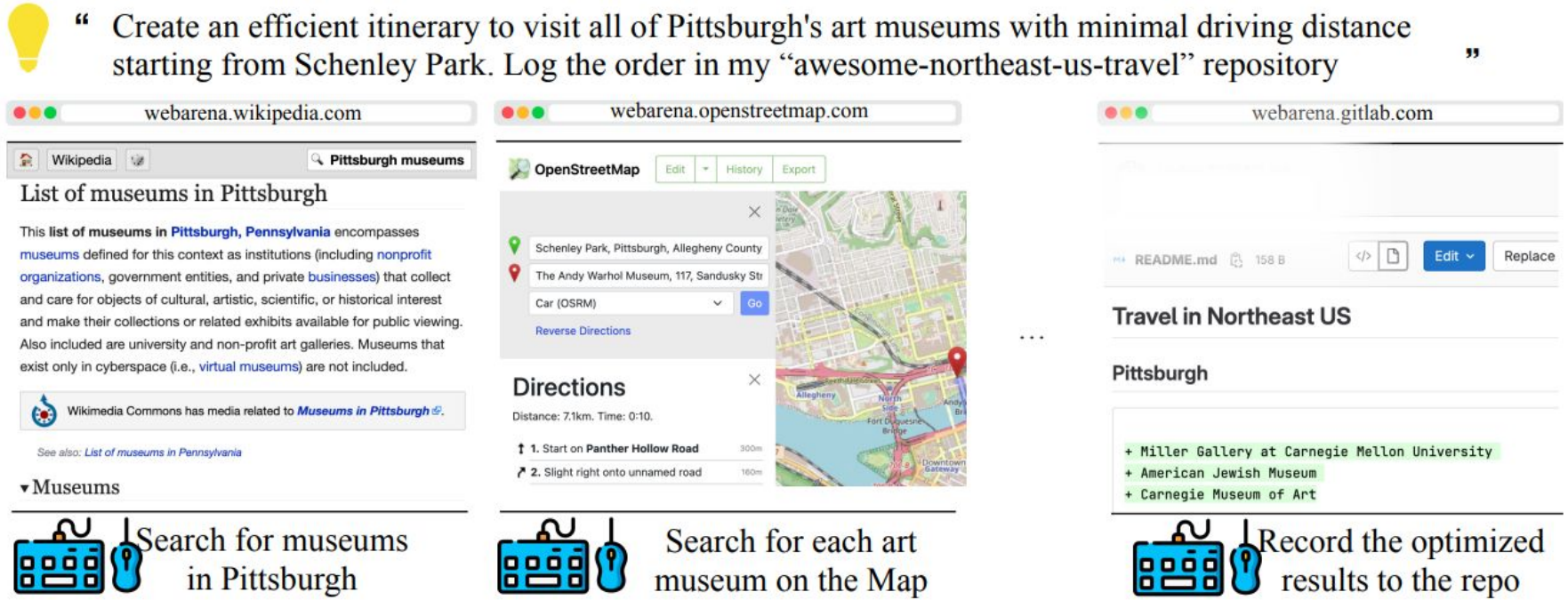


Figure 2: A high-level task that can be fully executed in `WebArena`. Success requires sophisticated, long-term planning and reasoning. To accomplish the goal (top), an agent needs to (1) find Pittsburgh art museums on Wikipedia, (2) identify their locations on a map (while optimizing the itinerary), and (3) update the README file in the appropriate repository with the planned route.

Website Categories in WebArena

Researchers analyzed 200 real browsing sessions.

Four major website categories were identified:

1. E-commerce platforms
    - Example: Amazon

2. Social discussion forums
    - Example: Reddit

3. Collaborative development platforms
    - Example: GitLab

4. Content Management Systems (CMS)

These represent common online activities.

Additional Utility Tools

WebArena also includes common online tools:

- Map → navigation and location searches
- Calculator → quick calculations
- Scratchpad → note-taking

Purpose:

- Support human-like problem solving.

Knowledge Resources

Information sources are included to support tasks.

Examples:

- General knowledge resources like Wikipedia
- Domain-specific manuals and documentation.

Agents can use these to research information before acting.

Implementation of Websites

WebArena uses open-source software to build websites.

Examples implemented:

- OneStopShop (E-commerce)
- GitLab platform
- Reddit-style discussion forum
- CMS platform
- Map service
- Wikipedia clone

Data was imported from real-world platforms.

Environment Reproducibility

The environment Design:

- Content extracted from real-world websites
- Hosted using Docker containers
- Uses OpenAI Gym style APIs

Benefits:

- Easy setup for researchers
- Identical environments across systems
- Scripts allow resetting to a fixed initial state

Observation Space

Observation space simulates a web browser interface.

Agents observe:

- Current URL
- Open browser tabs
- Content of the active webpage

Important feature:

- Supports multiple tabs, like real browsing.

Page Content Representations

WebArena provides page information in multiple formats:

1. HTML DOM tree
    - Structure of web page elements.
2. Screenshot
    - Pixel image of webpage.
3. Accessibility tree
    - Simplified structured representation of elements.

Accessibility trees:

- Smaller and easier for models to process.

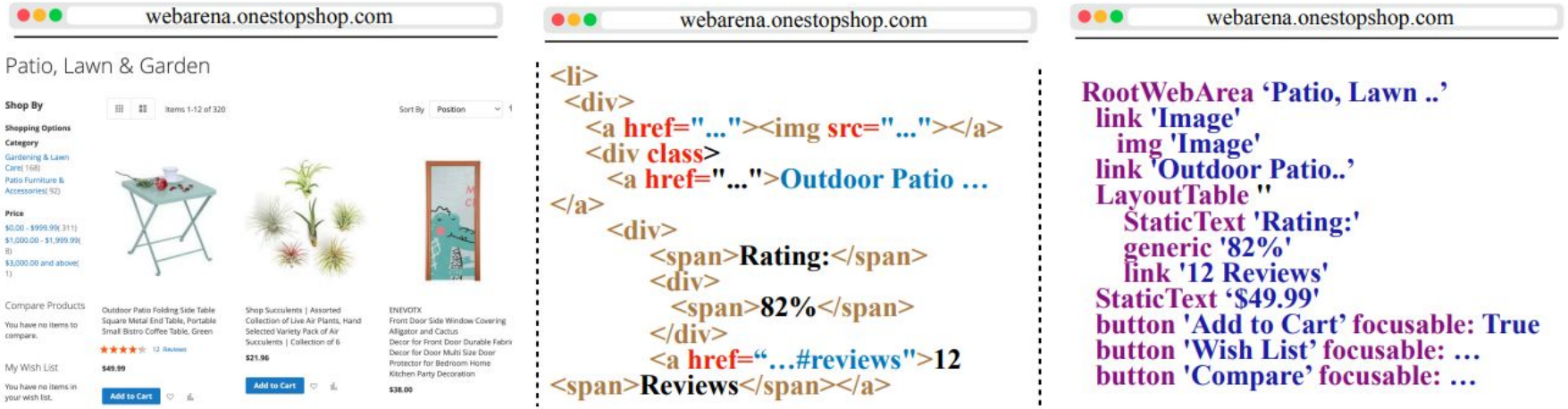


Figure 3: We design the observation to be the URL and the content of a web page, with options to represent the content as a screenshot (left), HTML DOM tree (middle), and accessibility tree (right). The content of the middle and right figures are trimmed to save space.

Viewport Limitation

Observation can be limited to the visible portion of the page.

Reason:

- Some AI models have limits on:
    - context length
    - image resolution

This helps models process only relevant information.

Action Space

Agents interact using keyboard and mouse-like actions.

Three action categories:

1. Element actions
    - click
    - hover
    - type
    - keyboard shortcuts

2. Tab actions
    - open tab
    - close tab
    - switch tab

3. Navigation actions
    - visit URL
    - forward/back navigation

Element Selection Methods

Agents can select web page elements in two ways:

1. Screen coordinates (x, y)
2. Unique element IDs

Example action: click [1582]

If element 1582 = Add to Cart, the agent clicks that button.

Advantages of Element IDs

Using element IDs:

- Simplifies element selection
- Avoids ambiguity
- Turns selection into a classification problem

Benefits:

- Works with different AI agent designs
- Maintains fair performance comparison metrics.

Benchmark Overview

- WebArena provides a benchmark with 812 web-based tasks.
- Tasks evaluate how well AI agents can:
    - Convert natural language instructions
    - Into real interactions with websites.
- Each task includes a metric to measure functional correctness.

Goal: Evaluate whether an agent actually completes the intended task.

Intent Collection

Tasks are based on high-level user intents.

Annotators:

- Explored WebArena websites first.
- Then created realistic tasks.

Key requirement:

- Tasks must involve multiple steps.

Example:

- Simple: *Click the science subreddit*
- Better: *Post a greeting message on the science subreddit*

Task Design Criteria

Annotators followed three main guidelines:

1. High-level tasks
    - Require several actions.

2. Creative tasks
    - Add constraints for complexity.

3. Template-based tasks
    - Use variables to create multiple versions.

Example template:

- “Create a {{site1}} account identical to my {{site2}} account”

Task Dataset Statistics

Dataset summary:

- 241 task templates
- 812 total task instances
- Average 3.3 variations per template

Annotators used ChatGPT to generate task ideas.

Task Categories

Tasks are divided into three main categories:

1. Information Seeking:

- Agent finds information and returns a text answer.

Example:

- “When was the last time I bought shampoo?”

Task Categories

2. Site Navigation

- Agent navigates webpages to locate information or sections.

Example:

- Find a specific page in a forum.

3. Content & Configuration Operations

- Agent modifies website content or settings.

Examples:

- Create a post
- Update documentation
- Make an online purchase

Action Type	Description
`noop`	Do nothing
`click(elem)`	Click at an element
`hover(elem)`	Hover on an element
`type(elem, text)`	Type to an element
`press(key_comb)`	Press a key comb
`scroll(dir)`	Scroll up and down
`tab_focus(index)`	focus on i-th tab
`new_tab`	Open a new tab
`tab_close`	Close current tab
`go_back`	Visit the last URL
`go_forward`	Undo `go_back`
`goto(URL)`	Go to URL

Figure 4: Action Space of `WebArena`

Category	Example
Information Seeking	When was the last time I bought shampoo
	Compare walking and driving time from AMC Waterfront to Randyland
Site Navigation	Checkout merge requests assigned to me
	Show me the ergonomic chair with the best rating
Content & Config	Post to ask "whether I need a car in NYC"
	Delete the reviews from the scammer Yoke

Figure 5: Example intents from three categories.

Evaluating Information Seeking Tasks

Three evaluation methods:

1. Exact Match
    - Predicted answer must match the correct answer exactly.

2. Must Include
    - Answer must contain specific key information.

3. Fuzzy Match
    - Semantic similarity evaluation using GPT-4.

    Example:

    - “2h58min” = “2 hours 58 minutes”.

Function	ID	Intent	Eval Implementation
$r_{\text{info}}(a^*, \hat{a})$	1	Tell me the name of the customer who has the most cancellations in the history	`exact_match`($\hat{a}$, "Samantha Jones")
	2	Find the customer name and email with phone number 8015551212	`must_include`($\hat{a}$, "Sean Miller") `must_include`($\hat{a}$, "sean@gmail.com")
	3	Compare walking and driving time from AMC Waterfront to Randyland	`fuzzy_match`($\hat{a}$, "walking: 2h58min") `fuzzy_match`($\hat{a}$, "driving: 21min")
$r_{\text{prog}}(\mathbf{s})$	4	Checkout merge requests assigned to me	`url=locate_current_url(s)` `exact_match(URL, "gitlab.com/merge_requests?assignee_username=byteblaze")`

	5	Post to ask "whether I need a car in NYC"	`url=locate_latest_post_url(s)`
			`body=locate_latest_post_body(s)`
			`must_include(URL, "/f/nyc")`
			`must_include(body, "a car in NYC")`

Table 1: We introduce two evaluation approaches. r_{info} (top) measures the correctness of performing information-seeking tasks. It compares the predicted answer $\hat{a}$ with the annotated reference a^* with three implementations. r_{prog} (bottom) programmatically checks whether the intermediate states during the executions possess the anticipated properties specified by the intent.

Evaluating Navigation & Operation Tasks

Evaluation checks actual website state.

Methods include:

- Database queries
- API calls
- JavaScript extraction from web pages

Example:

- Verify whether a forum post was created correctly.

Unachievable Tasks

Some tasks are intentionally impossible to complete.

Example:

- Asking for contact information not available on a website.

Expected agent behavior:

- Respond with “N/A”.

Purpose:

- Test whether agents avoid making false claims.

Annotation Process

Task creation involved:

- Authors with experience in web tasks.
- External annotators.

Quality control:

- Each task annotated twice.
- Disagreements resolved by a third annotator.

Evaluation scripts were written using JavaScript.

Human Performance

Human evaluation:

- 5 computer science graduate students
- 170 tasks tested

Results:

- Average completion time: 110 seconds
- Overall success rate: 78.24%

Avg. Time	110s
$\text{Success Rate}_{\text{info}}$	74.68%
$\text{Success Rate}_{\text{others}}$	81.32%
$\text{Success Rate}_{\text{all}}$	78.24%

Common Human Errors

Failures often occurred due to:

- Misinterpreting task instructions
- Incomplete answers
- Partial task execution

Some failures involved navigating incorrect pages.

Baseline Web Agents Overview

- Researchers tested baseline AI agents in WebArena.
- Experiments used three large language models (LLMs).
- Two prompting strategies were evaluated.
- Each prompt included two example demonstrations.

Goal:

- Measure how well LLMs can perform web tasks from natural language instructions.

Prompting Strategy 1 - Direct Action Prediction

In the first approach:

- The LLM directly predicts the next action.

Input provided to the model:

- Task intent (natural language instruction)
- Current webpage observation
- Previous action history

Example output:

- `click [1582]`

Meaning:

- An actual click of the webpage element with ID: 1582.

Prompting Strategy 2 - Chain-of-Thought Reasoning

Second approach uses Chain-of-Thought (CoT) reasoning.

Process:

1. Model explains its reasoning steps in text.
2. Then predicts the next action.

Purpose:

- Encourage step-by-step problem solving.
- Improve decision making in complex tasks.

Agent Instructions and Environment Description

Before task examples, the prompt provides:

- Description of the browser environment
- List of allowed actions
- Important interaction rules

Purpose:

- Help the model understand how the environment works.

These instructions were similar to those given to human annotators to ensure fair comparison.

Handling Unachievable Tasks

Agents were informed that:

- Some tasks may not be possible to complete.

Instruction provided:

- If the agent believes a task is impossible, it should stop execution.

This instruction is called: Unachievable Hint (UA hint).

Purpose:

- Prevent agents from making incorrect assumptions or hallucinations.

Observation Format for Agents

Agents receive webpage information as:

- Accessibility tree representation
- Includes unique element IDs

Each webpage element contains:

- Role (button, link, etc.)
- Text content
- Properties

Example element:

- `[1582] Add to Cart`

Performing Actions on Web Elements

Agents interact with elements using their IDs.

Example command: click [1582]

Result:

- Clicks the Add to Cart button.

Benefits of using IDs:

- Removes ambiguity
- Simplifies element selection
- Makes interaction easier for AI models.

Experiment Setup

Experiments involved:

- Three LLM models
- Two prompting strategies
- Two demonstration examples per prompt

Additional details such as:

- Prompt design
- Model configuration

Are provided in the paper’s appendix.

WebArena Results Overview

Evaluated GPT-4, GPT-3.5, TEXT-BISON-001 on WebArena tasks.

Tasks involve multi-step, realistic web interactions.

Human performance: 78.24% success

GPT-4 with CoT: 11.70% success

GPT-3.5 with CoT: 8.75%

TEXT-BISON-001: 5.05%

Highlights challenges of long-horizon tasks in realistic environments.

CoT	UA Hint	Model	SR	SR_{AC}	SR_{UA}
✓	✓	TEXT-BISON-001	5.05	4.00	27.78
✗	✓	GPT-3.5	6.41	4.90	38.89
✓	✓	GPT-3.5	8.75	6.44	58.33
✓	✓	GPT-4	11.70	8.63	**77.78**
✗	✗	GPT-3.5	5.10	4.90	8.33
✓	✗	GPT-3.5	6.16	6.06	8.33
✓	✗	GPT-4	**14.41**	**13.02**	44.44
-	✓	Human	78.24	77.30	100.00

Table 2: The end-to-end task success rate (SR %) on `WebArena` with different prompting strategies. **CoT**: the model performs step-by-step reasoning before issuing the action. **UA hint**: ask the model to stop when encountering unachievable questions.

Error Analysis – Early Stopping

Models often stop prematurely, thinking tasks are impossible.

GPT-4 mislabels 54.9% of achievable tasks as impossible due to Unachievable hint (UA hint).

Removing the hint improves GPT-4 success rate to 14.41%, but slightly reduces detection of truly unachievable tasks (44.44%).

GPT-3.5 struggles with hallucinations, repeated actions, or exceeding step limits.

Consistency Across Task Templates

Tasks from the same template can have different difficulties.

GPT-4 achieves 100% success on only 4 of 61 templates.

GPT-3.5 never achieves full completion on any template.

Example:

- Simple: “Fork metaseq”
- Complex: “Fork all repos from Facebook”

Indicates need for memory-based or strategy-reuse methods.

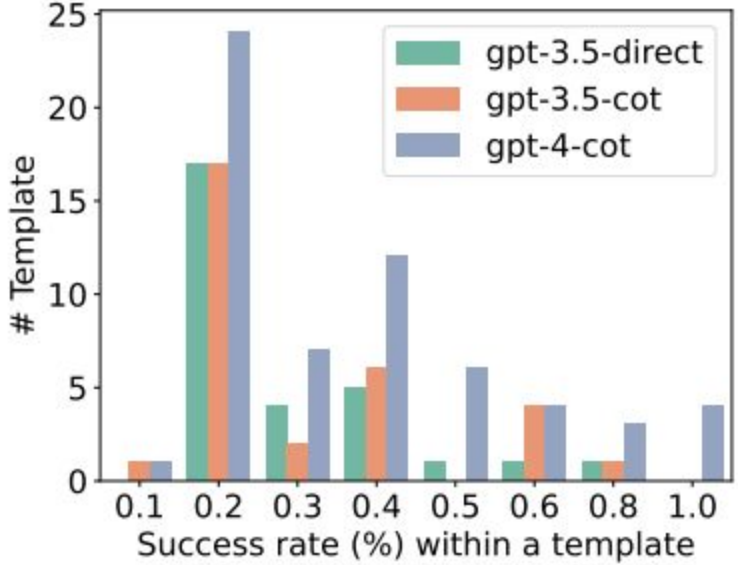


Table 3: Distribution of success rates on templates with ≥ 1 successful executions on GPT models (no UA hint).

Key Insights

Even state-of-the-art LLMs struggle with realistic web tasks.

Prompt design (e.g., UA hint) greatly affects performance.

Task variability shows AI inconsistency across similar templates.

WebArena provides a benchmark for testing advanced AI strategies, including memory and reasoning enhancements.

Related Work – Benchmarks for Web Agents

Agents controlled via natural language have been studied extensively.

Existing benchmarks often face trade-offs:

- Static states → limited exploration & functional evaluation
- Simplified environments → reduced task variety
- Live apps (e.g., AndroidEnv) → less reproducible

Games and other simulations often diverge from real-world objectives.

Benchmark		Dynamic Interaction?	Realistic Environment?	Diverse Human Tasks?	Functional Correctness?
Mind2Web	(Deng et al., 2023)	✗	✓	✓	✗
Form/QAWoB	(Shi et al., 2017)	✗	✓	✓	✗
MiniWoB++	(Liu et al., 2018)	✓	✗	✗	✓
Webshop	(Yao et al., 2022a)	✓	✗	✗	✓
ALFRED	(Shridhar et al., 2020)	✓	✗	✗	✓
VirtualHome	(Puig et al., 2018)	✗	✗	✓	✗
AndroidEnv	(Toyama et al., 2021)	✓	✓	✗	✗
WebArena		✓	✓	✓	✓

Table 4: The comparison between our benchmark and existing benchmarks on grounding natural language instructions to concrete executions. Our benchmark is implemented in our fully interactable highly-realistic environment. It features diverse tasks humans may encounter in their daily routines. We design evaluation metrics to assess the functional correctness of task executions.

Related Work – Interactive Decision-Making

Advanced agents incorporate:

- Hierarchical planning → break tasks into sub-tasks
- Error recovery and self-correction
- Multi-modal inputs → screenshots or DOM trees
- Task as programs → enable skill reuse and structured planning

Techniques like search, backtracking, observation summarization improve robustness.

WebArena challenges these methods due to complex, realistic web environments.

Conclusion

WebArena is a standalone, realistic, reproducible environment.

Includes fully functional web apps with organic data from popular domains.

Curated benchmark of 812 tasks mapping high-level natural language to web actions.

Outcome-based evaluation validates true task success.

Key Findings & Takeaways

Even GPT-4 achieves only 14.41% success, vs humans at 78.24%.

Highlights current limitations of AI in realistic web tasks.

WebArena provides a testbed for developing robust autonomous agents.

Future research should focus on:

- Improving planning, reasoning and memory
- Enhancing error recovery and task generalization

Q & A

MIND2WEB: Towards a Generalist Agent for the Web

Xiang Deng* Yu Gu Boyuan Zheng Shijie Chen Samuel Stevens Boshi Wang Huan Sun* Yu Su*

MIND2WEB: Towards a Generalist Agent for the Web

- Research on building AI agents that can operate on real websites
- Introduces a large-scale dataset for training web-interaction agents
- Proposes a method (MINDACT) combining small and large language models
- Goal: Enable AI systems to perform complex web tasks from natural language instructions

Abstract Overview

- The paper introduces MIND2WEB, a dataset for training generalist web agents
- Contains 2,000+ tasks from 137 real-world websites across 31 domains
- Provides annotated interaction traces showing how users complete tasks
- Demonstrates how large language models can interact with web environments
- Shows that filtering web page elements before LLM reasoning improves performance

The Vision: Generalist Web Agents

- A generalist web agent can perform tasks on any website
- Receives a natural language instruction from the user
- Navigates pages and interacts with elements like a human
- Performs multi-step tasks involving clicks, typing, and selections
- Ultimately completes the user’s goal autonomously

Example Web Agent Workflow

- User provides a task instruction
- Agent interprets the goal using natural language understanding
- Agent navigates across multiple web pages
- Performs actions such as clicking buttons or filling forms
- Completes the final objective (e.g., booking, searching, purchasing)

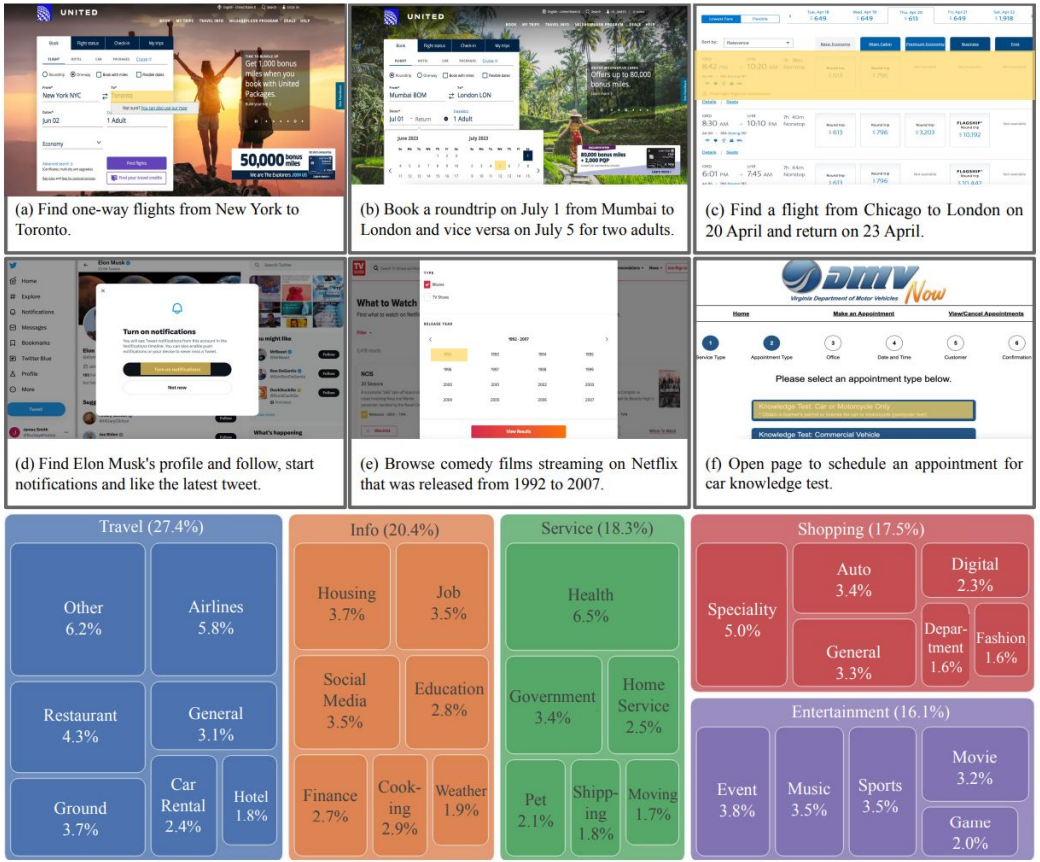


Figure 1: Sample tasks and all domains featured in MIND2WEB. The array of diversity allows for testing an agent's generalizability across tasks on the same website (a vs. b), similar tasks on different websites (a vs. c), and even to entirely disparate tasks, websites, and domains (d−f).

Why This Problem Matters

- Many online tasks require navigating complex web interfaces
- Web automation tools require technical programming skills
- AI assistants could make the web more accessible
- Agents could automate repetitive online tasks
- The web could become a universal interface for AI systems

Key Requirements for a Generalist Web Agent

- Must operate across many different websites
- Must generalize to unseen websites and domains
- Must handle large and complex HTML structures
- Must perform multi-step reasoning and interactions
- Must understand natural language instructions

Challenges of Real Web Environments

- Webpages often contain thousands of HTML elements
- Layouts and interfaces vary widely between websites
- Websites are dynamic and change frequently
- Agents must plan sequences of actions
- Only the high-level goal is given, not step-by-step instructions

Limitations of Prior Work

- Many web agent datasets use simulated environments
- Others only cover a small number of websites
- Tasks are often simplified or scripted
- Agents often rely on predefined workflows
- These limitations prevent building generalist agents

Research Goal

- Develop a benchmark for generalist web interaction agents
- Use real-world websites instead of simulations
- Provide diverse tasks and domains
- Enable evaluation of generalization to unseen websites
- Facilitate research on AI agents that can interact with the real web

Key Contributions of the Paper

- Introduces the MIND2WEB dataset for web agents
- Provides 2,000+ tasks with detailed interaction traces
- Proposes the MINDACT framework for web interaction
- Evaluates LLM-based agents across different generalization settings
- Releases dataset and models to support future research

The MIND2WEB Dataset

- A large-scale dataset for training web interaction agents
- Contains 2,350 verified tasks
- Collected from 137 real websites
- Covers 31 different domains such as travel, shopping, and services
- Designed to test realistic web navigation abilities

Dataset Design Principles

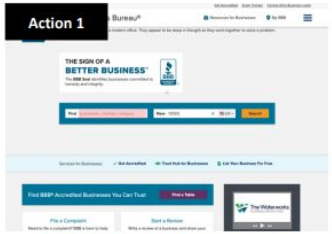
- Use real websites rather than simulated environments
- Include diverse tasks and domains
- Capture complete interaction sequences
- Represent realistic user behavior on websites
- Support research on generalization

Task Description:
Show me the reviews for the auto repair business closest to 10002.

Action Sequence:

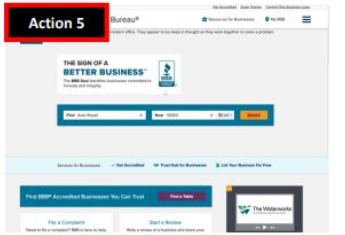
	Target Element	Operation
1.	[searchbox] *Find*	**TYPE:** ***auto repair***
2.	[button] *Auto Repair*	**CLICK**
3.	[textbox] *Near*	**TYPE:** ***10002***
4.	[button] *10002*	**CLICK**
5.	**[button] *Search***	**CLICK**
6.	[switch] *Show BBB Accredited only*	**CLICK**
7.	[svg]	**CLICK**
8.	[button] *Sort By*	**CLICK**
9.	**[link] *Fast Lane 24 Hour Auto Repair***	**CLICK**
10.	[link] *Read Reviews*	**CLICK**

Webpage Snapshots:

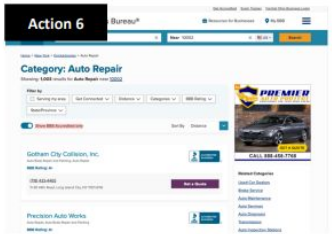


<input name="find_text" type="search">

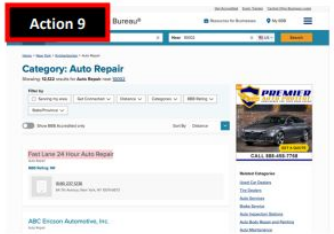
Auto Repair



<button>Search</button>



<button>Show BBB Accredited only</button>



Fast Lane 24 Hour Auto Repair

Read Reviews

Figure 2: A sample data instance of our dataset with the three components. Actions marked in **red** will result in a transition to a new webpage.

Types of Web Interactions

Agents interact with web pages using three main operations:

- Click: press buttons or links
- Type: input text into fields
- Select: choose options from dropdown menus

These actions mimic typical human web behavior; Complex tasks require multiple steps

Task Structure in MIND2WEB

Each task contains three components:

- Task description – natural language instruction
- Action sequence – ordered steps to complete the task
- Webpage snapshots – environment data for replaying interactions
- Tasks may span multiple webpages
- Interaction traces capture the complete task execution

Webpage Environment Representation

The dataset stores webpage environments using several formats:

- MHTML snapshots containing full HTML content
- DOM trees describing web page structure
- Screenshots of webpages
- HAR files capturing network traffic
- These enable offline replay of web environments

Data Collection Pipeline

The dataset was created using a structured pipeline:

- Website selection
- Task proposal
- Task demonstration
- Task verification

This ensures tasks are realistic and high quality.

Website Selection

- 137 popular websites selected
- Cover 31 domains including travel, retail, and services
- Chosen to represent diverse web interfaces
- Ensures agents encounter varied interaction patterns
- Provides broad coverage of real-world web usage

Task Proposal

- Annotators generate realistic tasks
- Tasks are inspired by seed prompts generated by language models
- Instructions describe high-level user goals
- Tasks are open-ended rather than scripted
- Example: “Find the cheapest flight to Paris”

Task Demonstration

- Annotators perform tasks on the websites
- A custom Playwright-based tool records interactions
- Each step records the target element and action
- Interaction traces form the ground truth demonstrations
- Ensures realistic sequences of user actions

Task Verification

- Researchers manually review demonstrations
- Remove unnecessary actions
- Improve task descriptions for clarity
- Ensure steps correctly complete the task
- Final dataset contains 2,350 high-quality tasks

Task Formulation for Learning

The agent receives:

- A task description
- The current webpage environment
- Previous interaction history

The agent must:

- Select the correct webpage element
- Predict the correct action
- Repeat until the task is completed

Table 1: Statistics of MIND2WEB compared with existing datasets.

	# Dom.	# Env.	Env. Type	Avg. # Elements	# Tasks	Task Info.	Avg. # Actions
MiniWoB++ [22]	–	100	Simplified mobile websites	28	100	Low-level	3.6
WebShop [40]	1	1	Simplified shopping websites	38	12,000 products	High-level	11.3
RUSS [39]	–	22	Real-world websites	801	80	High & low	5.4
PixelHelp [21]	4	4	Mobile apps	–	187	High & low	-
META-GUI [35]	6	11	Mobile apps	79	1,125 dialogues	High-level	4.3
MoTIF [5]	15	125	Mobile apps	188	756	High & Low	4.4
MIND2WEB	5 / 31	137	Real-world websites	1,135	2,350	High-level	7.3

Core Research Challenge

The main challenge is **grounding language instructions into webpage actions**.

This requires the agent to:

• Understand natural language goals

• Interpret web page structure

• Identify relevant elements

• Execute correct interactions

The Problem with Raw Webpages

- Webpages often contain **thousands of HTML elements**
- LLM context windows cannot process entire pages
- Many elements are irrelevant to the task
- Efficient filtering is necessary
- This motivates the **MINDACT architecture**

Overview of MINDACT

MINDACT is a two-stage framework for web interaction.

Stage 1:
- Small language model selects candidate elements

Stage 2:
- Large language model predicts the final action

This reduces complexity and improves efficiency.

Two-Stage Architecture

Step 1 — Candidate Generation
- Filter web page elements

Step 2 — Action Prediction
- Choose the correct element and action

This pipeline allows LLMs to focus only on relevant webpage elements.

Stage 1: Candidate Generation

- Uses a small language model
- Each DOM element is represented using text and attributes
- Model scores relevance of elements to the task
- Training uses positive and negative element examples
- Outputs **top-k candidate elements**

Candidate Generation Model

- Uses **DeBERTa-base (86M parameters)**
- Produces ranked list of candidate elements
- Top 50 elements selected for next stage
- Significantly reduces webpage complexity
- Enables efficient processing by LLMs

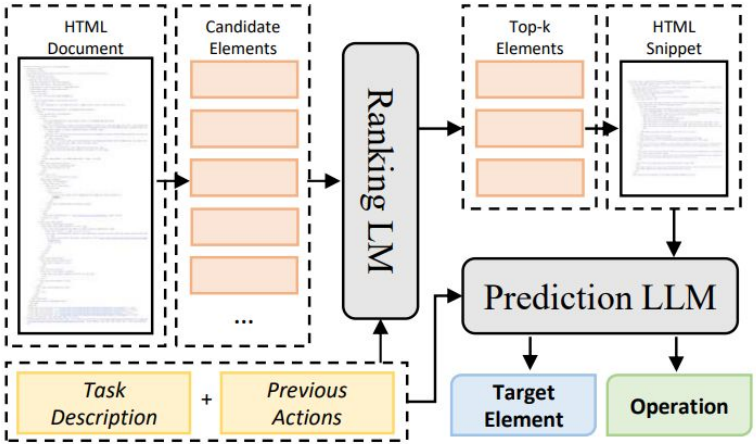


Figure 3: The overall pipeline for MINDACT with a small ranking LM for candidate generation, and a large prediction LM for action prediction.

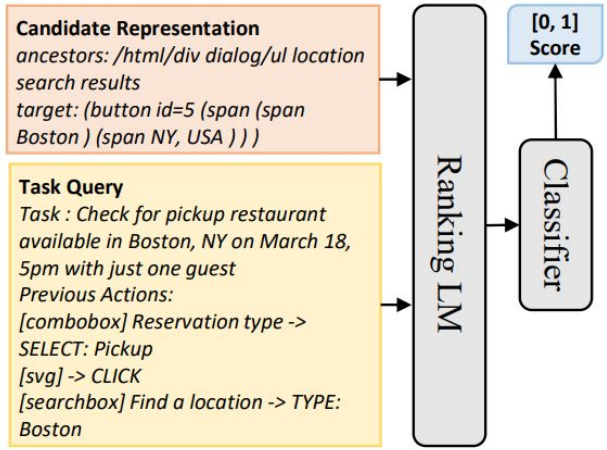


Figure 4: Illustration of the candidate generation module and the templates for constructing task query and candidate representation.

Stage 2: Action Prediction

- Uses a large language model
- Input contains task description and candidate elements
- Predicts which element to interact with
- Predicts the operation type
- Predicts any required value (e.g., typed text)

Multi-Choice Action Prediction

Action prediction is formulated as:

Multiple-choice question answering

The model chooses:

- Which element to interact with
- Which operation to perform
- Whether none of the candidates are correct

Iterative Candidate Selection

- Candidates are grouped into small sets
- The LLM evaluates each set sequentially
- If none are correct, the model selects “None”
- Process continues until the correct element is found
- Improves accuracy when many candidates exist

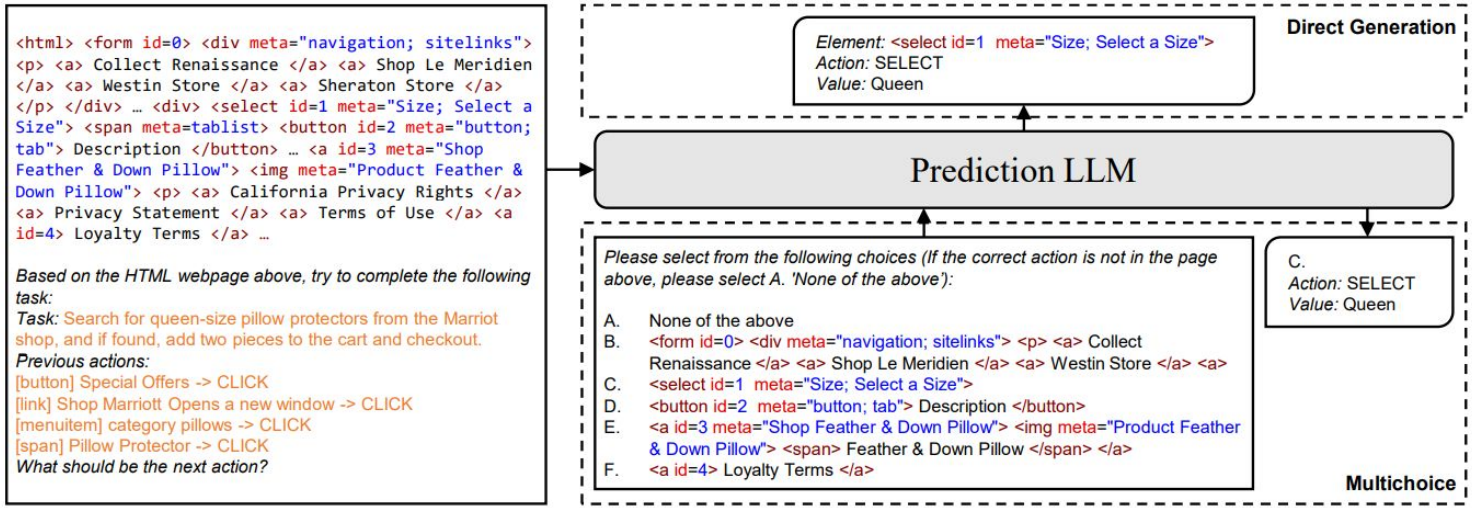


Figure 5: Illustration of action prediction with LLMs.

Table 2: Main results. The classification baseline uses $\text{DeBERTa}_{\text{B}}$ and the generation baseline uses $\text{Flan-T5}_{\text{B}}$. For step-wise metrics, we report macro average across tasks. * For GPT-4 we use 50 tasks for each setting with top-10 candidates due to limited budget. See Appendix D.3 for results on the 50 tasks subsets for all methods.

	Cross-Task				Cross-Website				Cross-Domain			
	Ele. Acc	**Op. F1**	**Step SR**	**SR**	**Ele. Acc**	**Op. F1**	**Step SR**	**SR**	**Ele. Acc**	**Op. F1**	**Step SR**	**SR**
Classification	26.8	–	–	–	21.6	–	–	–	24.5	–	–	–
Generation	20.2	52.0	17.5	0.0	13.9	44.7	11.0	0.0	14.2	44.7	11.9	0.4
MINDACT												
w/ $\text{Flan-T5}_{\text{B}}$	43.6	76.8	41.0	4.0	32.1	**67.6**	29.5	1.7	33.9	**67.3**	31.6	1.6
w/ $\text{Flan-T5}_{\text{L}}$	53.4	**75.7**	50.3	**7.1**	39.2	67.1	35.3	1.1	39.7	67.2	37.3	2.7
w/ $\text{Flan-T5}_{\text{XL}}$	**55.1**	**75.7**	**52.0**	5.2	**42.0**	65.2	**38.9**	**5.1**	**42.1**	66.5	**39.6**	**2.9**
w/ GPT-3.5	20.3	56.6	17.4	0.8	19.3	48.8	16.2	0.6	21.6	52.8	18.6	1.0
w/ GPT-4*	41.6	60.6	36.2	2.0	35.8	51.1	30.1	2.0	37.1	46.5	26.4	2.0

Training Setup

- Models trained using ground-truth demonstrations
- Training uses **left-to-right language modeling**
- Each step corresponds to predicting the next action
- Previous actions are included as context
- Enables learning sequential decision-making

Experimental Setup

Experiments evaluate:

- Ability to complete web tasks
- Generalization to new websites
- Performance of different models

Experiments use **three evaluation settings**.

Generalization Settings

Three evaluation scenarios are used:

- **Cross-Task** – new tasks on familiar websites
- **Cross-Website** – unseen websites in known domains
- **Cross-Domain** – entirely new domains

These tests measure true generalization ability.

Cross-Task Evaluation

- Same websites as training data
- New tasks not seen during training
- 252 evaluation tasks
- 69 websites involved
- Tests ability to learn task reasoning

Cross-Website Evaluation

- Websites not seen during training
- Same domains as training data
- 177 tasks used
- Tests ability to adapt to new website layouts

Cross-Domain Evaluation

- Completely new domains
- Includes information and service websites
- 912 tasks across 73 websites
- Most challenging evaluation setting

Evaluation Metrics

Four metrics measure performance:

- Element Accuracy – correct element selected
- Operation F1 – accuracy of action type
- Step Success Rate – element and action both correct
- Task Success Rate – entire task completed correctly

Strict Evaluation Criteria

- Tasks require multiple steps
- A single mistake causes task failure
- Therefore task success rate is much lower than step accuracy
- Provides a realistic evaluation of web agents

Candidate Generation Results

Recall@50 performance:

- Cross-Task: 88.9%
- Cross-Website: 85.3%
- Cross-Domain: 85.7%

This shows the filtering stage reliably keeps the correct element.

Baseline Methods

The study compares several baselines:

- Small language model alone
- Autoregressive LLM generation (Flan-T5)
- Multi-choice QA formulation (MINDACT)

These help evaluate the effectiveness of the proposed method.

Action Prediction Results

Best results achieved with MINDACT:

- Step Success Rate (Cross-Task): 52%
- Cross-Website: ~39%
- Cross-Domain: ~39%

Shows moderate success but significant room for improvement.

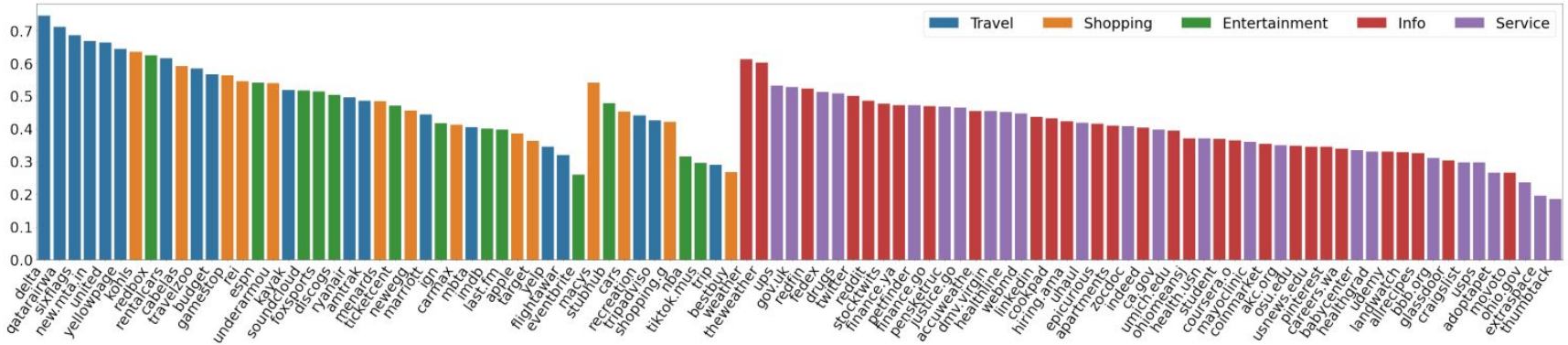


Figure 6: Step success rate per website grouped by the three splits: $\text{Test}_{\text{Cross-Task}}$, $\text{Test}_{\text{Cross-Website}}$ and $\text{Test}_{\text{Cross-Domain}}$ from left to right. Here we only show websites with more than three test tasks.

Table 7: Step Success Rate for all methods on the 50 tasks subsets we used to evaluate GPT-4. Numbers in parentheses are the results on the full test set (same as Table 2)

	Cross-Task	**Cross-Website**	**Cross-Domain**
Flan-T5$_{B}$	43.3 (41.0)	25.3 (29.5)	28.1 (31.6)
Flan-T5$_{L}$	48.1 (50.3)	30.8 (35.3)	27.6 (37.3)
Flan-T5$_{XL}$	47.9 (52.0)	33.3 (38.9)	34.6 (39.6)
GPT-3.5	15.2 (17.4)	15.1 (16.2)	16.7 (18.6)
GPT-4	36.2	30.1	26.4

Insights from Results

- Models perform best on familiar websites
- Performance drops on unseen websites
- Cross-Website and Cross-Domain scores are similar
- Main challenge is website layout diversity

In-Context Learning with GPT Models

Experiments with large models:

- GPT-3.5 achieves ~20% element accuracy
- GPT-4 performs similarly to fine-tuned models
- Large models show promise for web agents
- However inference cost is high

Key Experimental Findings

- Two-stage filtering significantly improves LLM efficiency
- Multi-choice reasoning improves action prediction
- Generalization remains difficult
- Real-world websites are extremely complex

Limitations of the Dataset

- Mostly English-language websites
- Many websites are U.S.-based
- Annotators are primarily MTurk workers
- Limited demographic diversity in task creation

Modeling Limitations

- Current model only uses textual HTML information
- Ignores visual layout cues such as buttons and icons
- Each webpage processed independently
- Dynamic webpage behavior is not modeled

Human-Agent Interaction Limitations

- Tasks are executed from a single instruction
- No interactive guidance from users
- Users cannot correct the agent during execution
- Real systems may require conversational interaction

Offline Evaluation Limitations

- Evaluation uses cached website snapshots
- Agents cannot explore alternative task paths
- Real websites change frequently
- Online evaluation could provide more realistic testing

Safety Considerations

Potential risks of generalist web agents:

- Performing sensitive actions (banking, purchases)
- Bypassing security measures
- Automating harmful activities
- Misuse for large-scale manipulation

Safeguards are necessary for safe deployment.

Future Research Directions

- Integrate multimodal inputs (HTML + screenshots)
- Train specialized web-focused language models
- Use reinforcement learning with live website feedback
- Improve planning for long-horizon tasks

Broader Impact

Generalist web agents could:

- Improve accessibility for users with disabilities
- Simplify complex web tasks
- Enhance AI assistants and productivity tools
- Enable new forms of human-computer interaction

Conclusion

- MIND2WEB introduces the first large-scale dataset for generalist web agents
- MINDACT demonstrates a practical LLM-based interaction framework
- Experiments show promising but limited performance
- Significant challenges remain in generalization and reasoning

Key Takeaways

- Real-world web environments are extremely complex
- Dataset diversity is critical for training web agents
- Two-stage filtering helps LLMs process large webpages
- Generalization to unseen websites remains a major challenge
- MIND2WEB provides a foundation for future research

Q & A